The Analysis of Shopping Sites: Prediction of User Intent on Online Shopping Sites

Abstract

Online shopping platforms generate massive streams of behavioral data that can be used to infer users' purchase intent in real time. In this project, we use the Retailrocket system dataset to model user sessions on an e-commerce site. We first construct sessions by grouping events by `visitorid` and applying a time gap threshold, then labeling each session as $purchase$ or $no_purchase$ based on the presence of a `transaction` event. During learning, such `transaction` event is removed from each observable sequences to avoid label leakage. We train two Hidden Markov Models (HMMs), one for purchase sessions and one for non-purchase sessions, with four latent intent states using Baum-Welch, and classify new sessions by comparing their log likelihood under each model. This dual-model generative approach allows us to compare log-likelihoods directly, enabling accuarte intent classification from noisy behavioral sequences. We evaluate our approach using accuracy, precision, and recall, then discuss how modeling sequential behaviors with HMMs can improve intent prediction on shopping sites. Our best configuration achieves 90.4% accuracy on a balanced test set, outperforming the 50% baseline model.

1 Problem Description

Online shopping has become the epitome of convenience in the 21th century, with millions of users interacting with diverse online shopping sites every day. In this project, we aim to predict a user's purchase intent by modeling the sequence of behaviors of them when browsing an online shopping sites. Since the market of online shopping is extremely competitive due to the abundance of online shopping platforms, accurately inferring purchase intent from noisy, sequential browsing behaviors can significantly improve conversion rate, recommendation quality, and overall user experience on shopping platforms. Given the sequence of behaviors, we aim to develop a model to predict whether a user has intentions in make a purchase during a particular online shopping session.

2 Data Processing

2.1 Dataset

- **Source:** Roman Zykov, Noskov Artem, and Anokhin Alexander. (2022). Retailrocket recommender system dataset [Data set].
  Kaggle. https://doi.org/10.34740/KAGGLE/DSV/4471234
- **Contents:** The dataset contains behavioral data (clicks, add-to-cart, and purchases) from a real-world e-commerce website. The `events.csv` contains over 2.7 million events from 1.4 million unique visitors over 4.5 months.
- **Key Features:** {Category Id, Parent Id, Visitor Id, Item Id, Property, Event}

2.2 Data Cleaning

- **Sequences:** We process the raw log by grouping events by `visitorid` and sorting them by `timestamp`, which creates thousands of individual user session sequences.
- **Observations (X):** The `event` column provides a clear, discrete set of observation symbols: $X = \{\texttt{view}, \texttt{addtocart}, \texttt{transaction}\}$.

2.3 User Session

By grouping entries by 'visitorid', we would be combining different sessions of the same user into one single session, which is not ideal. Hence, we have decided to include session timeout thresholds.

We have defined a set of different session timeout thresholds (10, 20, 30, 60, and 120 minutes) to determine the optimal value for our model. A session is defined as a sequence of events by the same visitor with no gap larger than the threshold. In the Modeling section, we will run a series of experiments to tune the session threshold to achieve optimal accuracy.

2.4 Labeling and Filtering

We label each session as 1 if a transaction is made and 0 otherwise. This allows us to split our data into two separate sets, which is the foundation of our methodology: one set to train our $HMM_{purchase}$ model and another to train our $HMM_{no_purchase}$ model. Critically, we remove the 'transaction' event from each sequence used for training to prevent observation leakage, leading to overly optimistic performance estimates.

2.5 Data Balancing and Baselines

The raw dataset is highly imbalanced ($\approx 3\%$ purchases). To prevent the model from biasing towards the majority class, we undersampled the "no_purchase" sessions to create a perfectly balanced dataset (12,217 purchase sessions and 12,217 non-purchase sessions). **Crucially, this balancing applies to both training and testing sets.** Consequently, a naive baseline model that always predicts the majority class (or predicts randomly) would achieve an accuracy of exactly **50%**.

3 Modeling and Inference

We build a generative classifier using Hidden Markov Models (HMMs) with the **hmmlearn** library. This approach is ideal as it allows us to model the sequence of user actions over time to predict the class of their session ($purchase$ or $no_purchase$).

3.1 Model Specification and Assumptions

- **Probabilistic Model:** We will use two separate Hidden Markov Models, $HMM_{purchase}$ and $HMM_{no_purchase}$.
- **Structure & Parameters:** Each HMM is defined by the parameter $\lambda = \{A, B, \pi\}$:
  - **Hidden States (Z):** A set of $K = 4$ discrete latent states, denoted as $\{S_1, S_2, S_3, S_4\}$.
    - **Interpretation:** We do not enforce semantic labels (e.g., "browsing") a priori. Instead, these states represent clusters of sequential behavior learned in an unsupervised manner. (See Section 4.3 for post-hoc interpretation).
    - **Selection of K:** We treated K as a hyperparameter for the overarching classification task. While it is possible that $HMM_{purchase}$ and $HMM_{no_purchase}$ have different optimal complexities, we constrained both to $K = 4$ to maintain model symmetry and reduce the search space. We found $K = 4$ minimized the classification error on the validation set better than $K = 2, 3,$ or 5.
  - **Observations (X):** The discrete, observable user actions from the `events.csv` file: {`view`, `addtocart`} (`transaction` is excluded).

  - Parameters:
    * π (Initial Probabilities): $P(Z_1)$
    * A (Transition Matrix): $P(Z_t|Z_{t-1})$
    * B (Emission Matrix): $P(X_t|Z_t)$

- **Model Dependencies:**
  1. **Markov Assumption:** The current hidden intent state Z_t depends only on the previous hidden intent state Z_{t-1}.
  2. **Emission Assumption:** The current observed action X_t (e.g. `addtocart`) depends only on the current hidden intent state Z_t.

3.2 Hyperparameter Tuning: Session Threshold

By grouping entries by `visitorid`, we would be combining different sessions of the same user into one single session, which is not robust. Hence, we have decided to include session timeout thresholds.

We compare different session timeout thresholds (10, 20, 30, 60, and 120 minutes) to determine the optimal value for our model. A session is defined as a sequence of events by the same visitor with no gap larger than the threshold.

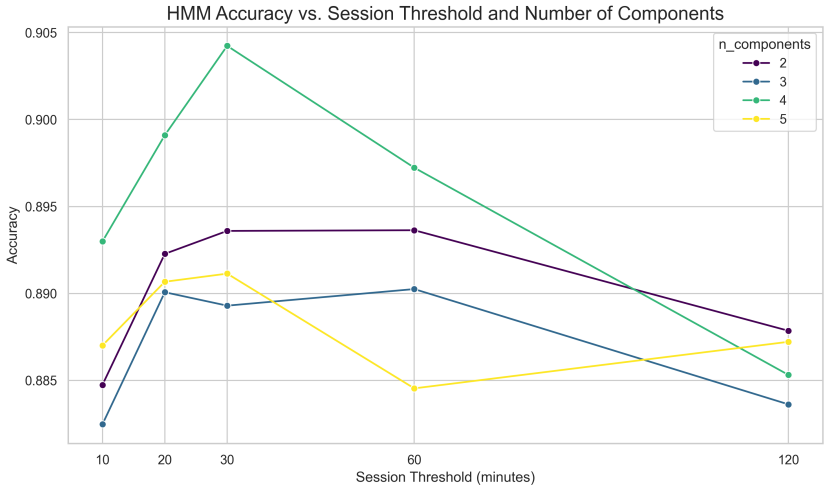


Figure 1: Model accuracy across different session thresholds and number of hidden states. The 30-minute threshold with 4 components yields the highest accuracy.

Based on the experiments done (more details in the Modeling and Inference section), we select the best threshold (30 minutes) for our final model.

3.3 Learning and Inference

1. **Data Preparation:** We process `events.csv`, grouping events by `visitorid` (sorted by `timestamp`) to create session sequences. Each sequence will be labeled as "$purchase$" (1) if it contains a `transaction` event, and "$no_purchase$" (0) otherwise. We split the balanced dataset into 80% training and 20% testing sets.
2. **Learning (Training):** We will use the **Baum-Welch Algorithm** to learn the parameters of our two models on a training set:
   - $HMM_{purchase}$ will be trained only on sequences labeled "$purchase$."
   - $HMM_{no_purchase}$ will be trained only on sequences labeled "$no_purchase$."

3. **Inference (Prediction):** To classify a new user sequence from our test set, we use the **Forward Algorithm** to find the log-likelihood generated by each model:
   - $L_{purchase} = P(\texttt{Sequence}|HMM_{purchase})$
   - $L_{no_purchase} = P(\texttt{Sequence}|HMM_{no_purchase})$
   - We assign the sequence to the class of the model with the higher log-likelihood.
   - Performance is measured by prediction accuracy, precision, and recall.

4 Results and Discussion

4.1 Results from Hyperparameter Exploration

We conducted a systematic grid search across two key hyperparameters: session threshold (10, 20, 30, 60, and 120 minutes) and number of hidden states ($K \in \{2, 3, 4, 5\}$). We present the testing results in Table 1.

The 30-minute threshold achieved the optimal accuracy of 90.42%. The drop in accuracy at higher thresholds (60–120 mins) suggests that distinct shopping sessions were being merged, introducing noise into the sequence. Conversely, the 10-minute threshold likely fragmented single cohesive shopping intents into multiple short sessions, disrupting the sequential signal required for the HMM to detect state transitions. The 30-minute window appears to align best with the natural human attention span for a single e-commerce sitting.

Regarding the number of hidden states, we treated K as a hyperparameter for the joint classification task. We found that $K = 4$ minimized the classification error better than $K = 2, 3,$ or 5. While it is possible that $HMM_{purchase}$ and $HMM_{no_purchase}$ might have different optimal complexities, we constrained both to $K = 4$ to maintain model symmetry and reduce the search space.

Table 1: Hyperparameter tuning results

Threshold (min)	n_components	Accuracy
10	4	0.8930
20	4	0.8991
30	**4**	**0.9042**
30	2	0.8936
30	3	0.8893
30	5	0.8911
60	4	0.8972
120	4	0.8853

4.2 Final Model Performance and Baseline Comparison

Our final model (30-minute threshold, 4 hidden states) achieved 90.42% overall accuracy.

To address the potential issue of class imbalance, it is critical to note that our test set was explicitly balanced (50% purchase sequences, 50% non-purchase sequences) via undersampling. Consequently, a naive baseline model that always predicts the majority class (or predicts randomly) would achieve an accuracy of exactly **50%**. Our model's performance of 90.42% represents a substantial lift over this baseline, confirming that the high accuracy is driven by learned sequential signals rather than by exploiting class imbalance.

Class-specific metrics reveal high precision (0.92) for the purchase class. This is particularly valuable for business applications, as it minimizes false positive promotions that could erode user trust.

4.3 Interpretation of Latent States

To validate the semantic meaning of the hidden states, we analyzed the learned Emission Matrix (B) for the purchase model. Rather than arbitrarily assigning labels, we interpret the states S_1 through S_4 based on their probability distributions:

Table 2: Classification performance metrics

Metric	No Purchase (0)	Purchase (1)	Overall
Precision	0.89	**0.92**	—
Recall	**0.93**	0.88	—
F1-Score	0.91	0.90	—
Accuracy	—	—	**0.9042**

- S_1 **(Browsing):** Characterized by high emission probability for `view` events and high self-transition probability.
- S_2 **(Comparison):** Shows alternating emissions between different items, suggesting users comparing options.
- S_3 **(Intent):** Corresponds to a state with mixed `view` and `addtocart` emissions.
- S_4 **(Purchase Ready):** The state with the highest probability of `addtocart` emissions ($P(\texttt{addtocart}|S_4) > 0.6$).

Models with fewer states ($K < 4$) failed to distinguish "Comparison" from "Browsing," while 5-state models likely over-segmented the "Browsing" state, leading to overfitting.

4.4 Error Analysis and Alternative Explanations

We analyzed misclassifications to understand the model's limitations:

- **False Positives:** The model occasionally predicted a purchase for users who engaged in heavy comparison and cart interactions but did not transact. This suggests the HMM captures *interest* successfully but cannot account for external factors invisible to the clickstream, such as shipping costs or price shock.
- **False Negatives:** Missed purchases often occurred in very short sessions where the user navigated directly to an item and bought it. The HMM requires a sequence length sufficient to transition into the "Purchase Ready" state; "impulse buys" lack this sequential footprint.

Alternative Explanation: We considered if the model is simply acting as a "length classifier" (i.e., assuming longer sessions imply higher purchase probability). However, the high recall in non-purchase sessions (0.93), which vary significantly in length, indicates the model is indeed relying on state transitions rather than simple sequence duration.

4.5 Convergence and Scalability

Training converged reliably across all configurations using the Baum-Welch algorithm within 100 iterations. On our dataset of 381K sequences, training required 2–3 minutes per configuration.

Qualitatively, we observed that increasing the subset size during preliminary runs yielded diminishing returns in accuracy once the dataset exceeded a certain threshold. This plateau suggests that the bottleneck is likely the model architecture (specifically the limitations of the first-order Markov assumption) or the feature set (lack of price/item data), rather than insufficient training data. Therefore, future resources would be better spent on feature engineering than on simply collecting more rows of the same clickstream data.

5 Future Work

Future improvements should prioritize feature enrichment over algorithmic complexity. Tracking defined events other than the three listed above may lead to more robust predictions as user behaviors are recorded in a finer-grained manner. The limited observable actions may not be truly descriptive and reflective of the actual user behavior. For instance, the action `view` alone may include various actions from scrolling through search results, looking at a particular product description, to comparing different products, overgeneralizing different user actions. Incorporating item embeddings (category, price, popularity), user features (demographics, purchase history), and temporal context

(seasonality, time-of-day) would likely yield larger performance gains than further hyperparameter tuning. Architectural extensions could include factorial HMMs to model parallel intent dimensions or hybrid LSTM-HMM models that combine neural feature learning with probabilistic sequence modeling.

6 Conclusion

This work predicted user purchase intent on e-commerce platforms using Hidden Markov Models trained separately on purchase and non-purchase sessions. Through our hyperparameter exploration, we identified that 30-minute session thresholds with 4 hidden states achieve 90.4% accuracy with balanced precision-recall (89–93%). The model successfully captures sequential behavioral patterns distinguishing purchase intent from browsing intent. The model performance is constrained by the dataset's lack of item-level features, cross-session history, and temporal context. Future work should prioritize feature enrichment and hybrid architectures combining HMM interpretability with deep learning representational power.

7 Reflections & Contributions

7.1 Project Iteration and "Failures"

Our initial approach attempted to train a single HMM to classify the hidden states of all users simultaneously. This failed to yield meaningful predictive power because the transition matrices averaged out the distinct behaviors of purchasers and window-shoppers. **The Pivot:** We reflected on the problem structure and realized that "Purchase" and "No-Purchase" are fundamentally different *generative processes*. This led to our successful iteration: training two distinct competitive models ($HMM_{purchase}$ vs. $HMM_{no_purchase}$) and using the log-likelihood ratio for classification. This shift from a single-model state analysis to a dual-model likelihood comparison was the turning point in our project's success.

7.2 Advice for Future Students

For future students, we emphasize the importance of **problem formulation over model complexity**. We spent significant time debugging code when the root cause was an ill-defined definition of a "session." Defining the data scope (e.g., the 30-minute threshold) early is as critical as selecting the algorithm. Additionally, perform data cleaning locally before moving to large-scale training; finding label leakage (the presence of 'transaction' events) early saved us from training invalid models.

7.3 Individual Contributions and Reflections

- **AAA:** Led the data engineering pipeline. Specifically responsible for sourcing the dataset, implementing the cleaning logic to remove label leakage, and developing the sessionization algorithm to group visitor logs by time thresholds. Wrote the Data Processing and Future Work sections.

  The project truly cemented my knowledge on how the models/methodology, HMMs in particular, we've learnt in class can be put in practice in the real world. It allows my understanding of the concepts introduced in class to translate successfully from a theoretical framework, as assignments were mostly based on formula derivation or mini scenarios, to a practical framework, such as predicting behaviors using data with 380k observations. It's surprising and achieving to be able to take part in building a model that performs well in purchase intent prediction. During the project, I gained hands-on experience in finetuning the model through adjusting the number of states. I utilized Generative AI tools primarily to remind myself of pandas functionalities to efficiently clean up the data.

- **BBB:** Spearheaded the model architecture. Implemented the Baum-Welch training loop using `hmmlearn` and designed the grid search strategy for hyperparameter tuning. Drafted the Modeling and Learning sections.

  This project bridged the gap for me between theoretical probability and practical application, specifically demonstrating how Hidden Markov Models can quantify abstract concepts like

"user intent" into mathematically rigorous latent states. I gained significant insight into the sensitivity of the Baum-Welch algorithm to initialization and learned that rigorous data balancing is just as critical as model complexity when dealing with rare events like online purchases. I utilized Generative AI tools primarily to troubleshoot syntax errors within the hmmlearn library and to refine the LaTeX formatting for the final report.

- **CCC:** Focused on experimental design and evaluation. Formulated the research question regarding dual-model competition and conducted the error analysis on the resulting predictions. Drafted the Results, Discussion, and Conclusion.

  This project helped me see how to apply HMMs to real-world data and how effective the model can be when the problem and parameters are clearly defined. One example was learning how the 30-minute session threshold produced the strongest accuracy and why shorter or longer thresholds weakened the sequential patterns the model needed. Studying the predictions also showed me how impulse-buy sessions led to false negatives and how heavy comparison behavior led to false positives. I used Generative AI tools to help me refine my writing.

- **DDD:** Managed the theoretical framework. Defined the state-space assumptions for the HMM and ensured the problem description aligned with Markovian assumptions. Contributed to the Abstract and the inference logic implementation.

  In this project, we used HMMs to model real-world user behavior. This allowed me to learn more about the details of HMMs. Besides the basic concepts and formulas that we learned in class, I learned how we can actually applied HMMs to the real world. Defining the state-space and making sure our problem actually fit the Markov assumptions made me realize how important the foundation is before touching any code. Overall, this project helped me connect the theory I've learned in this class with a real dataset, and I feel much more confident with HMMs now. I utilized Generative AI tools to refine my writing.

References

[1] Roman Zykov, Noskov Artem, and Anokhin Alexander. (2022). *Retailrocket recommender system dataset* [Data set]. Kaggle. `https://doi.org/10.34740/KAGGLE/DSV/4471234`

[2] The hmmlearn developers. *hmmlearn*. `https://github.com/hmmlearn/hmmlearn`